UCS3461 Foundations of Artificial Intelligence

STRUCTURE OF AGENTS

Semester 4, B.E. Computer Science and Engineering
22 December 2025

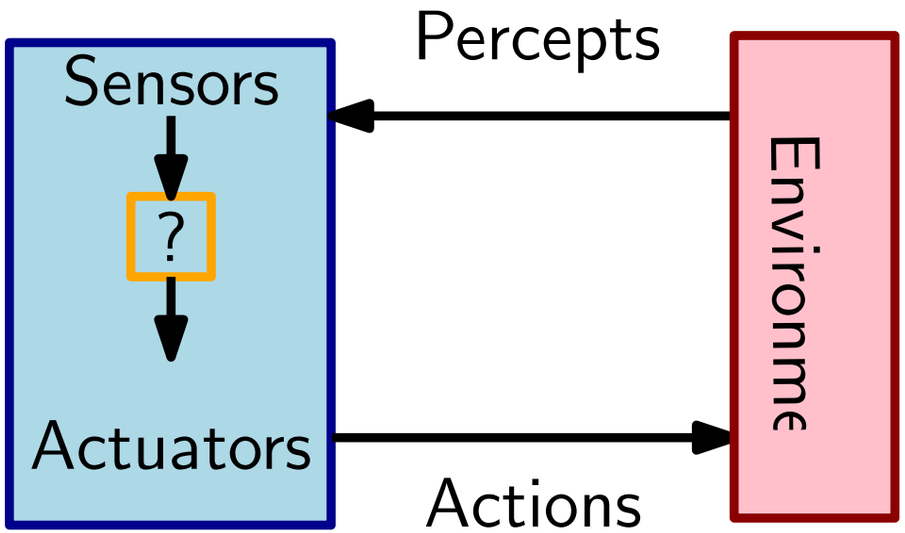
Sakaya Milton R
SSN College of Engineering

Recap

- Agents are situated in environment.
- Agents receive stimuli and do actions.
- Rational agents aim at maximizing the expected utility.
- Nature of environment decides the design of the agent.
- Agent = agent body/architecture + agent controller/program
- Agent controller: input – percept, output – command
- Belief state of an agent at time t encodes all of the agent's history.
- Controller implements agent function: agent's history $\rightarrow$ command
  - belief state function: belief state, percept $\rightarrow$ next belief state.
  - command function: belief state, percept $\rightarrow$ command.

Agents and Environments

- An agent perceives its environment through sensors and acts upon it through actuators
- Percepts: agent's perceptual inputs at any point in time.
- A rational agent chooses actions to maximize the expected utility
- Agent function describes an agent's behavior – maps any given percept sequence to an action
- Agent program implements agent function.

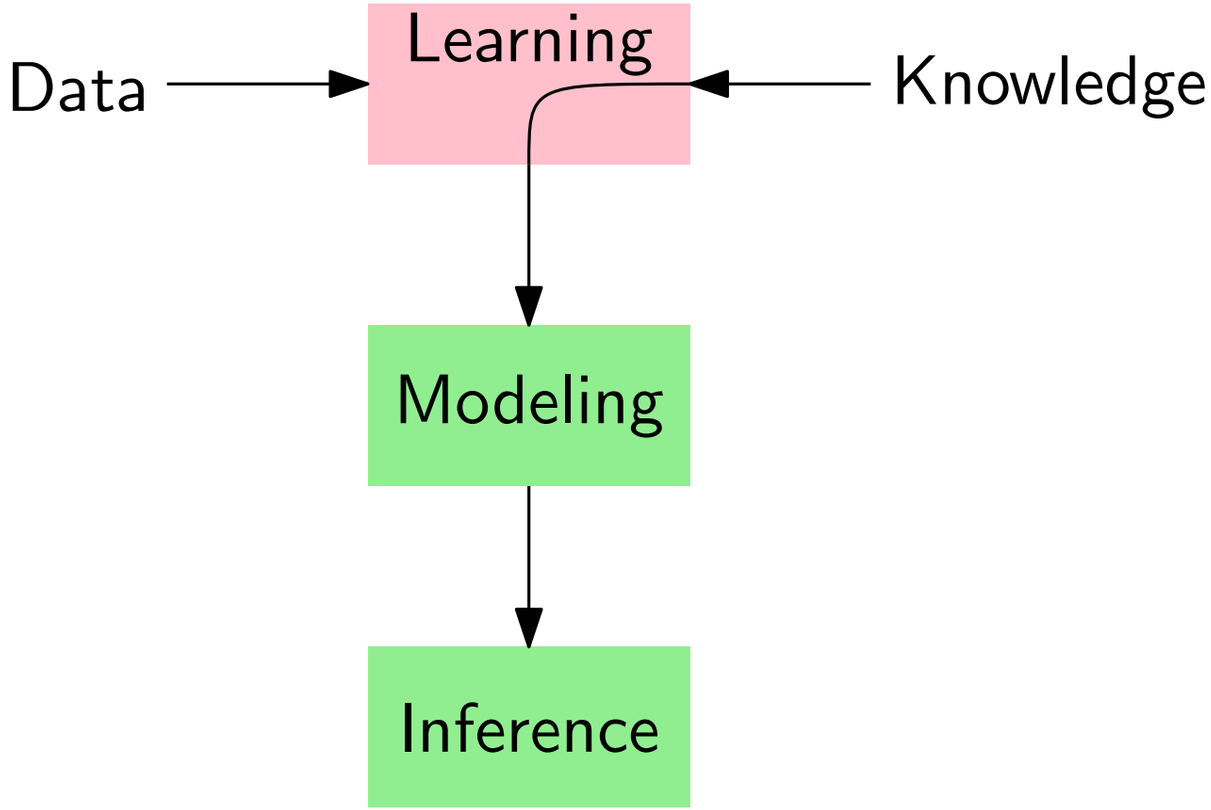


Properties of Task Environment

Dimensions of complexity

- Modularity: flat, modular, hierarchical
- Time axis: discrete, continuous
- Planning horizon: episodic, sequential
- Representation: states, features, relations
- Computational limits: perfect rationality, bounded rationality
- Learning: knowledge is given, knowledge is learned
- Sensing uncertainty: fully observable, partially observable
- Effect uncertainty: deterministic, stochastic
- Preference: goals, complex preferences
- Number of agents: single agent, adversary, multiple agents
- Interaction: offline, online

A Useful Paradigm



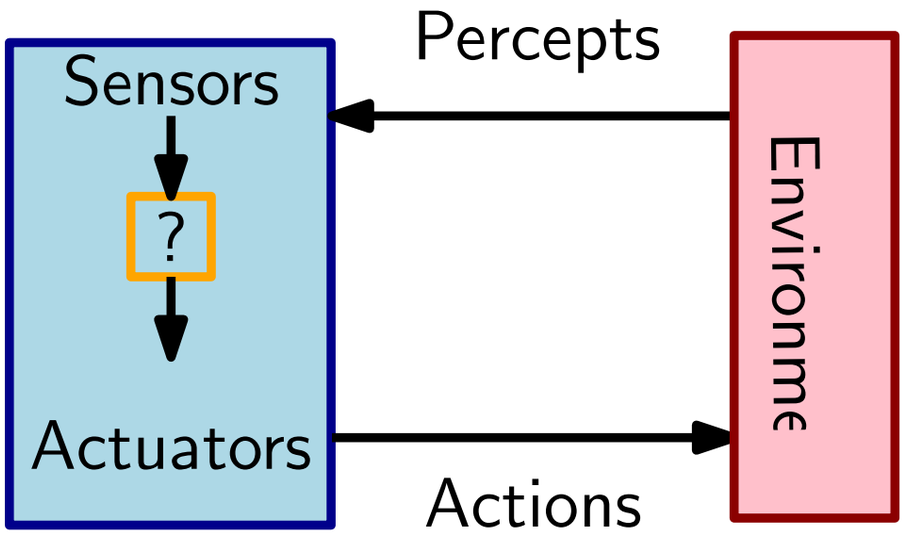
Agent Architecture

- Agent program implements the agent function – the mapping from percepts to actions.
- Architecture/body: computing device with physical sensors and actuators
- Architecture/body makes the percepts from the sensors available to the program, runs the program, and feeds the program's action choices to the actuators as they are generated.

  agent = architecture/body + program/controller

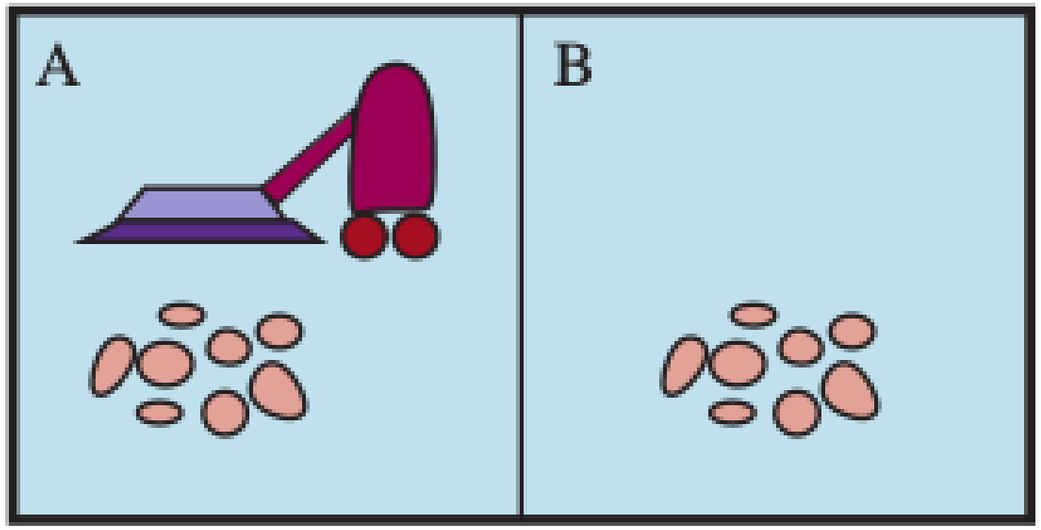
Agent Program

- Implements agent function, mapping percept sequences to actions
- Input: current percept from the sensors
- Output: an action to the actuators
- Agent program takes its input from the environme and hence the current percept as input
- Agent function takes the entire percept history as input
- If the action depends on the entire percept sequence, agent has to remember the percepts.



Example: Vacuum Cleaner Agent

- Percepts
  - Location: left, right
  - Cleanness: dirty, clean
- Action
  - Suck
  - Move left
  - Move right
- Agent function



Percept sequence	Action
(A, clean)	right
(A, dirty)	suck
(B, clean)	left
(B, dirty)	suck
(A, clean), (A, clean)	right
(A, clean), (A, clean), (A, dirty)	suck

Table-Driven Agent

Algorithm: table_driven_agent (percept)

Input: percept
Output: action
Global: percepts, a sequence (initially empty)
table, a table of actions, indexed by percept sequences (fully specified)

append percept to percepts
action ← look_up (table, percepts)
return action

- Space?
- Time?
- Know all entries?

Key AI challenge: Rational behavior from a smallish program is efficient rather than a vast table.

Space and Time

$$\text{Table size } = \sum_{t=1}^{T} |P|^t$$

where

P set of possible percepts
T life-time of the agent (total number of percepts it will receive)

- Chess is a tiny, well-behaved fragment of the real world – it has at least 10^{150} entries.
- The number of atoms in the observable universe is less than 10^{80}

1. No physical agent has the space to store the table,
2. The designer would not have time to create the table,
3. No agent could ever learn all the right table entries from its experience,
4. Even if the environment is simple enough to yield a feasible table size, the designer still has no guidance about how to fill in the table entries.

Types of Agents

Types of Agents

- Simple reflex agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents
- Learning agents

Reflex Agent Example: Vacuum Agent

- Percept (not the percept history) decides the action
- Vacuum agent: entries from 4^T to just 4.
- Condition-action rules

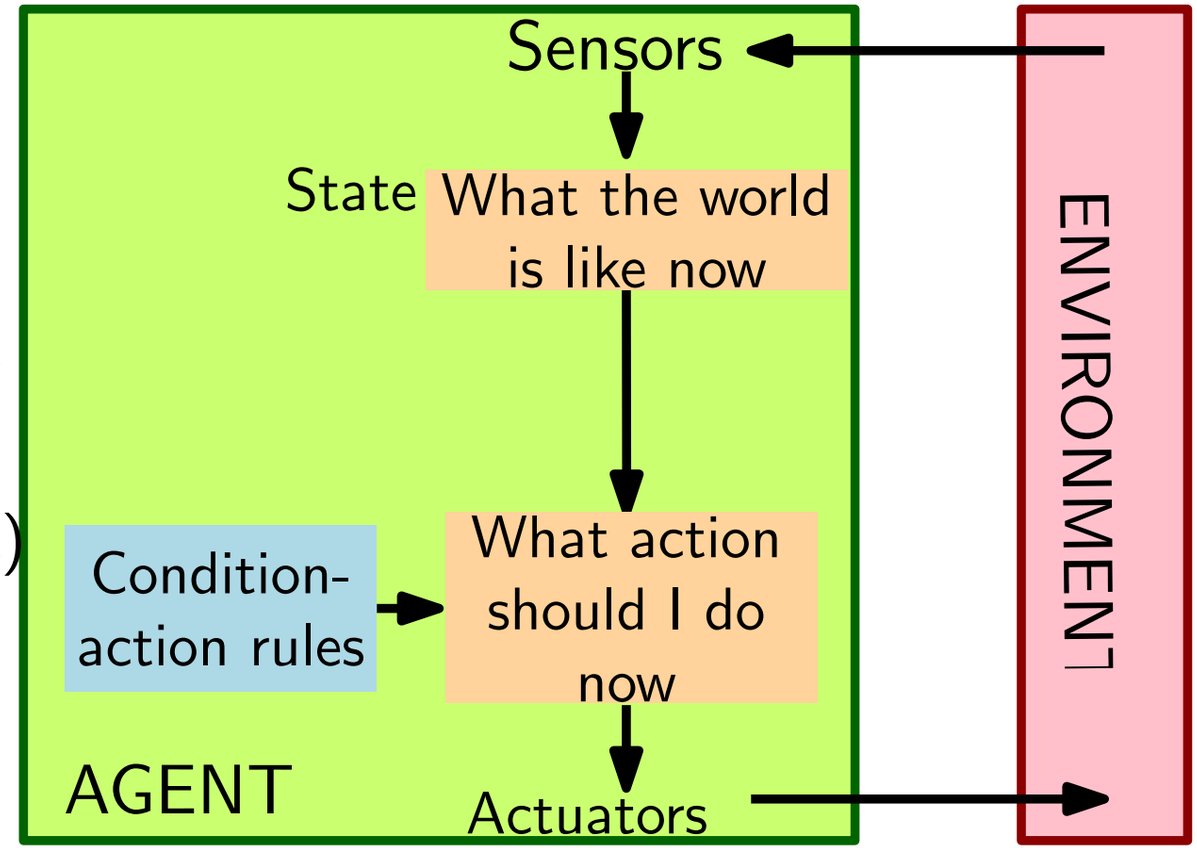
Algorithm: reflex_vacuum_agent ([location, status])

Input: [location, status]
Output: action

if status = Dirty **then**
    return Suck
else if location = A **then**
    return Right
else if location = B **then**
    return Left

Simple Reflex Agent

- If current percept is sufficient for deciding the action
- Only if the environment is fully observable
- Sensors fail? A little unobservable?



Algorithm: simple_reflex_agent (percept)

Input: percept
Output: action
Global: rules, a set of condition-action rules

state ← interpret_input (percept)
rule ← rule_match (rules, state)
return rule.action

Acts according to a rule whose condition matches the current state, as defined by the percept.

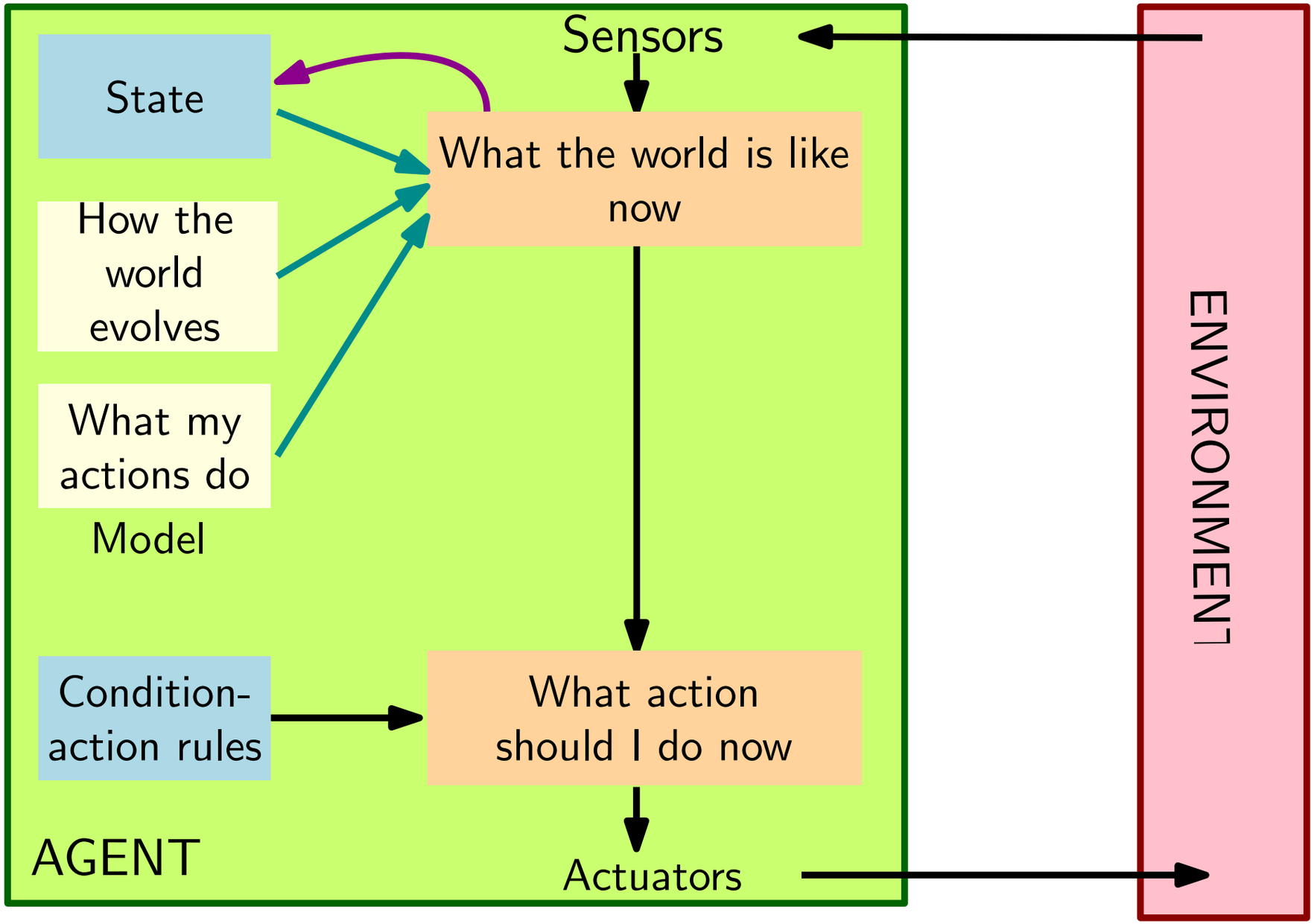
Model-based Reflex Agents

- The world is partially observable
- Agent keeps track of the state of the world, now unobserved.
- Belief state (internal state) of the agent: compactly represents percept history
- Agent updates the internal state – requires knowledge of
  - How the world evolves independently of the agent
  - How the agent's actions affect the world
- Model of the world: knowledge of "how the world works"
  - How the next state depends on current state and action

Model-based Reflex Agents

Keeps track of the current state of the world, using an internal model – chooses an action in the same way as the reflex agent.

Model-based Reflex Agents



Rarely possible to determine the current state of a partially observable environment exactly

Model-based Reflex Agents

- How models and states are represented.
- Agent can "guess" the current state of a partially observable environment – cannot determine exactly

Algorithm: model_based_reflex_agent (percept)

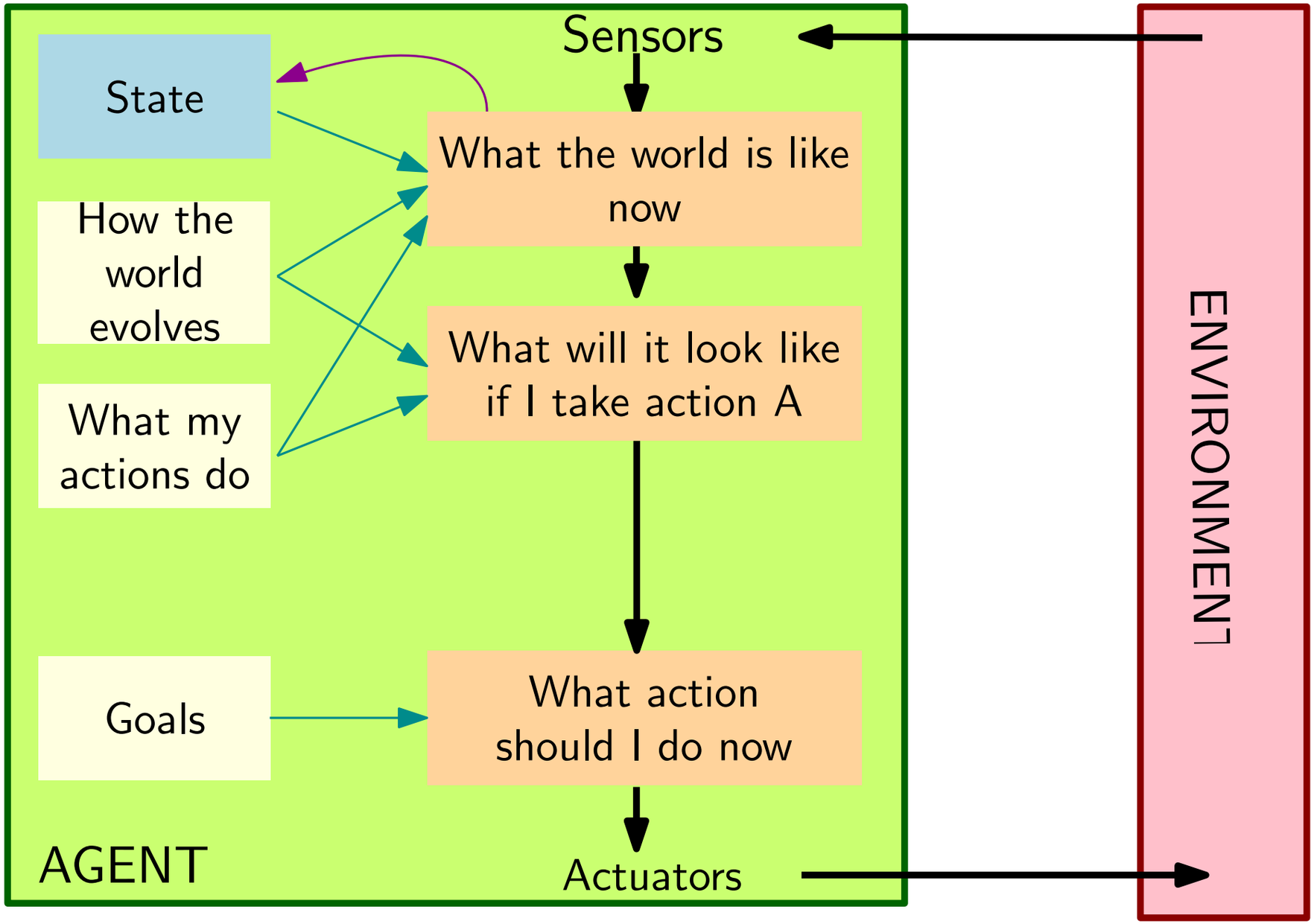
Input: percept
Output: action
Global: state, agent's current belief of the world state
model, how the next state depends on current state and action
rules, a set of condition–action rules
action, the most recent action, initially none

rule ← rule_match (rules, state)
action ← rule.action
state ← update_state (model, state, action, percept)
return action

Goal-based Agents

- Model + Goal (what is desirable)
- Search, planning: find action sequences that achieve the agent's goals
- More flexible: decision-supporting knowledge is represented explicitly and can be modified
- Reflex agent: we have to rewrite many condition-action rules.

Goal-based Agents

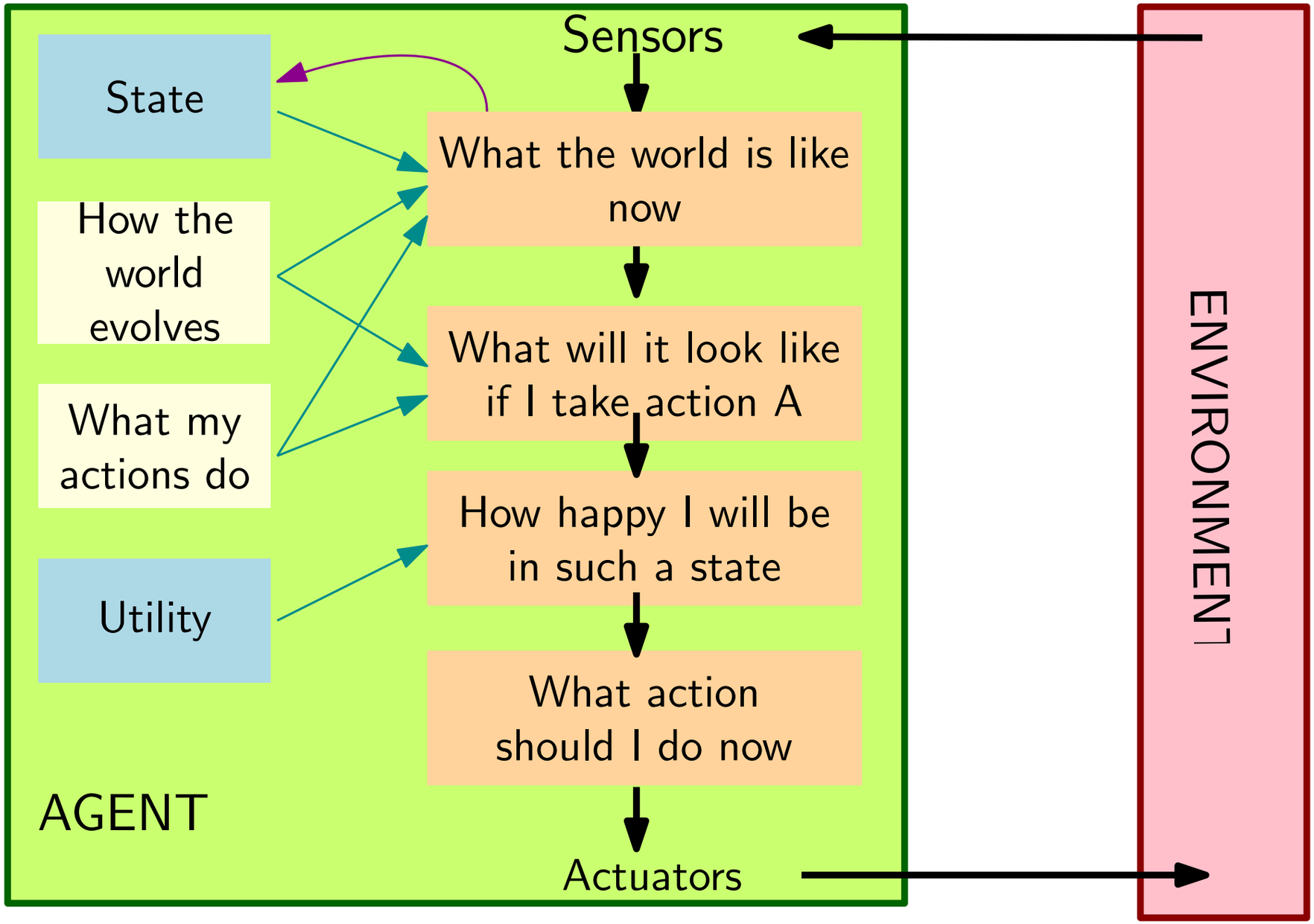


Utility-based Agents

- Goals: binary (reached or not reached?)
- Many action sequences may reach the goal
- Agent's utility function: internal representation of performance measure
- Inadequacy of goals (in goal-based agents)
  - Conflicting goals: utility function specifies the tradeoff.
  - Several goals with uncertainty in reaching – partial observability and stochasticity are ubiquitous in real world – utility connects likelihood and importance of goals.

Rational utility-based agent chooses the action that maximizes the expected utility of the action outcomes

Utility-based Agents



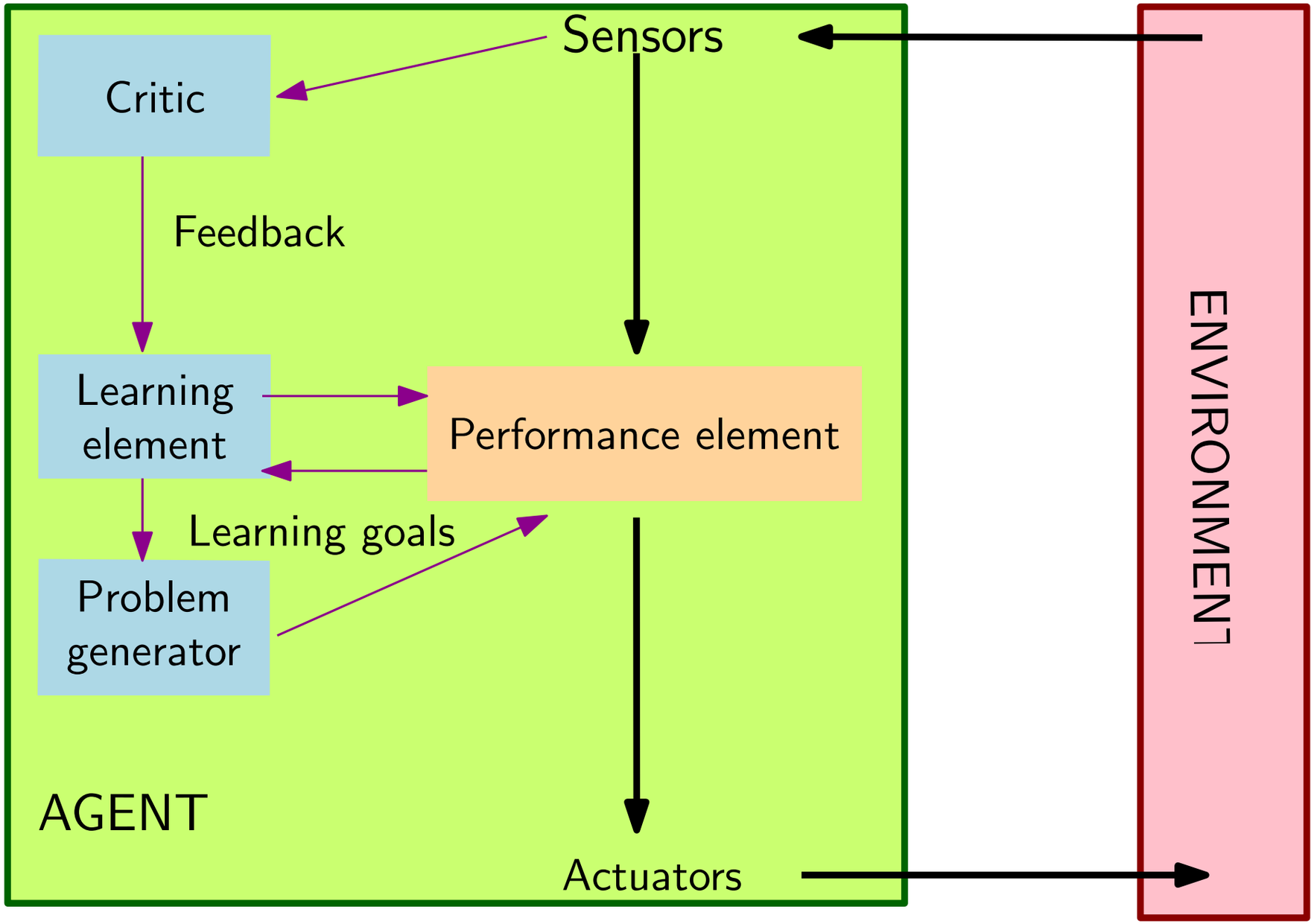
Utility-based Agents: General-Purpose Algorithm

- Any rational agent must behave as if it possesses a utility function
- If the utility function is explicit, the agent can make rational decisions with a general-purpose algorithm
- Does not depend on the specific utility function
- "Global" rationality $\Rightarrow$ a "local" constraint that can be expressed in a simple algorithm.
- A utility-based agent has to
  - model and keep track of its environment – research on perception, representation, reasoning, and learning
  - choose the utility-maximizing course of action – requires ingenious algorithms

Learning Agents

- Performance element (till now the entire agent)
  - receives percept
  - selects action
- Learning element — improves action selection
- Feedback from critic — how well the agent is doing according to fixed performance metric, and how the performance element should be modified to do better
- Problem generator suggests actions
  - exploit: optimal till now
  - explore: suboptimal, may discover better actions
- Learning element can make changes to any of the "knowledge" components of agent architecture (percept sequence, state changes, outcome sequence)

Learning Agents



Learning Agents

Learning in intelligent agents

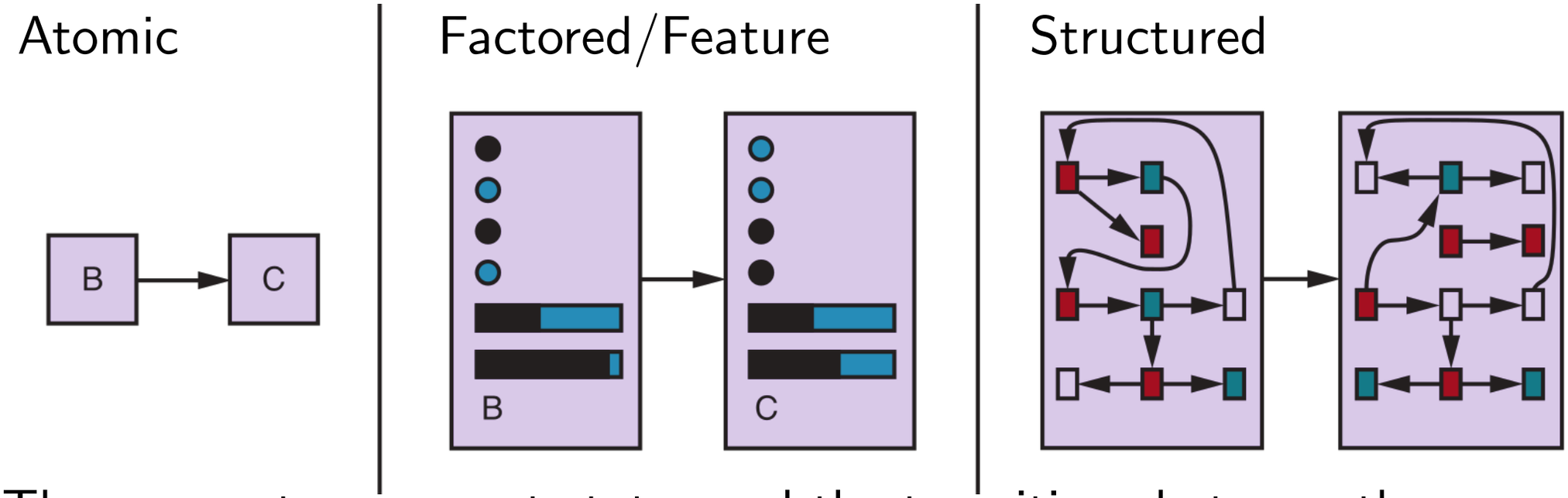
A process of modification of each component of the agent to bring the components into closer agreement with the available feedback information, thereby improving the overall performance of the agent.

Agent Components

Agent Components

- Agent components answer
  - What is the world like now?
  - What action should I do now?
  - What do my actions do?
- How components represent the environment.
- Representations along an axis of increasing complexity and expressive power: atomic, factored, and structured.

Agent Component Representation



Three ways to represent states and the transitions between them.

- Atomic representation: a state (such as B or C) is a black box with no internal structure
- Factored/Feature/Proposition representation: a state consists of a vector of attribute values (Boolean, real-valued, or one of a fixed set of symbols)
- Structured representation: a state includes objects, each of which may have attributes of its own as well as relationships to other objects.

Atomic Representation

- Each state of the world is indivisible – it has no internal structure.
- A single atom of knowledge, a "black box"
- Identical to or different from another black box.
- Search, game-playing, Hidden Markov Models (HMM), Markov Decision Process (MDP) – treat representations as if they were atomic.

Factored Representation

- Splits up each state into a fixed set of variables or attributes, each of which can have a value.
- Two different atomic states have nothing in common – they are just different black boxes
- Two different factored states can share some attributes and not others
- Makes it much easier to work out how to turn one state into another.
- We can also represent uncertainty – represented by leaving that attribute blank.
- Constraint satisfaction algorithms, propositional logic, planning, Bayesian networks, machine learning algorithms

Structured Representation

- World has things in it that are related to each other, not just variables with values.
- Relational databases, first-order logic, first-order probability models, knowledge-based learning, natural language understanding
- Almost everything that humans express in natural language concerns objects and their relationships.

Expressiveness vs Reasoning

- The more expressive a language, the more concise
- Example: rules of chess
  - can be written in one or two pages of a structured-representation language such as first-order logic
  - require thousands of pages when written in a factored-representation language such as propositional logic.
- On the other hand, reasoning and learning become more complex as the expressive power of the representation increases.
- To gain the benefits, intelligent systems for the real world may need to operate at all points along the axis simultaneously.

Intelligence Spectrum

Low-level intelligence

Reflex

States →
- Search problems
- Adversarial games
- Markov Decision Processes (MDP)

Variables →
- Constraint Satisfaction Problems (CSP)
- Bayesian networks

Logic →
- Relational

High-level intelligence

Summary

- Simple reflex agents
  - Current percept
- Model-based reflex agents
  - Current percept
  - State of the agent
  - Model of the world
- Goal-based agents
  - Current percept
  - State of the agent
  - Model of the world
  - Goal
- Utility-based agents
  - Current percept
  - State of the agent
  - Model of the world
  - Goal
  - Utility of the action's outcomes
- Learning agents
  - Improve with experience

End